



Lecturer: Flemming Christensen

INSTITUT FOR ELEKTRO- OG COMPUTERTEKNOLOGI, AARHUS UNIVERSITET

Email: flech@ece.au.dk

Litterature:

<https://www.probabilitycourse.com>

Chapter 14

Analysis versus Computer Simulation

- **Goal of simulation:** Use a computer program to approximate real-world probabilistic behavior when analytical calculation is difficult or impossible.
- Simulation quality depends on:
 - Accuracy of the model
 - Number of repetitions (law of large numbers)

What is a model?

In a simulation, the *model* is the set of assumptions and rules you use to generate data.

- Coin toss → assume $P(H) = 0.5$
- Queue system → assume arrival times follow a Poisson process
- Noise → assume Gaussian distribution

Example: Supermarket Checkout Simulation

Goal

A supermarket wants to understand:

- How long customers wait in line
- How many checkout counters they need

Customer arrivals: Assume arrivals follow a Poisson process

→ customers arrive randomly, but with an average rate (e.g., 20 per hour)

Using Python for Simulation

Importing Python modules

In this chapter we use NumPy, SciPy, and Matplotlib (Pyplot) for numerical computation and plotting.

```
import numpy as np
import scipy as sp
import matplotlib.pyplot as plt
```

Random number generator

We can create a random number generator **object** with NumPy:

```
rng = np.random.default_rng()
```

And then we call different **methods** on that generator:

```
rng.random()
# Uniform(0,1). Fixed bounds. Base primitive used internally by many methods.

rng.uniform(5, 10)
# Uniform(5,10). User-defined interval. General uniform distribution.

rng.normal(0, 1)
# Normal distribution with mean=0 and standard deviation=1.

rng.poisson(3)
# Poisson distribution with rate  $\lambda = 3$ . Returns counts of events.

rng.binomial(n=10, p=0.3)
# Binomial distribution: number of successes in n trials with prob p.
```

Uniform(0,1) for any distribution

Example direct method: Generate 10 normal random numbers with mean 0 and var 1

```
import numpy as np
import matplotlib.pyplot as plt
import scipy as sp

rng = np.random.default_rng()
x = rng.normal(0,1,10)
```



0	-1.27768
1	2.31082
2	0.116845
3	1.40987
4	-0.569258
5	0.360461
6	1.12287
7	1.35737
8	-1.53526
9	-1.50203

Uniform(0,1) is the *fundamental* behind all random sampling

Every built-in random generator (normal, Poisson, exponential, etc.) is internally produced using **Uniform(0,1)** values.

Generating Discrete Distributions from Uniform(0,1)

Example 1. Simulate tossing a coin with probability of heads p :

To simulate a coin toss with probability of heads p :

- Generate $U \sim \text{Uniform}(0, 1)$.
- Define

$$X_i = \begin{cases} 1, & U_i < p \\ 0, & U_i \geq p \end{cases} \quad \text{for } i = 1, \dots, n$$

Because

$$P(X = 1) = P(U < p) = p,$$

this produces a **Bernoulli(p)** random variable.

A Binomial random variable counts how many successes occur in n independent **Bernoulli(p)** trials:

$$X = X_1 + X_2 + \dots + X_n,$$

where each

$$X_i = \begin{cases} 1, & \text{with probability } p, \\ 0, & \text{with probability } 1 - p. \end{cases}$$

Generating Discrete Distributions from Uniform(0,1)

Python implementation with 5 tosses - counts how many heads occurred:

```
rng = np.random.default_rng()

n = 5
p = 0.5

U = rng.random(n)
toss = (U < p)

count_successes = 0
for i in range(n):
    count_successes = count_successes + int(toss[i])

print("Binomial sample:", count_successes)
```

count_successes	int	1	2
i	int	1	4
n	int	1	5
p	float	1	0.5
rng	random._generator.Generator	1	Generator object of numpy.random._generator module
toss	Array of bool	[5]	[False True True False False]
U	Array of float64	[5]	[0.95040193 0.33211674 0.34982733 0.86335755 0.86258792]

Generating Discrete Distributions from Uniform(0,1)

Example 2. Repeated Coin Toss Simulation (Law of Large Numbers)

Compute the running proportion of heads after i tosses. Repeat 5 times:

```
N = 1000
rng = np.random.default_rng()
p = 0.5

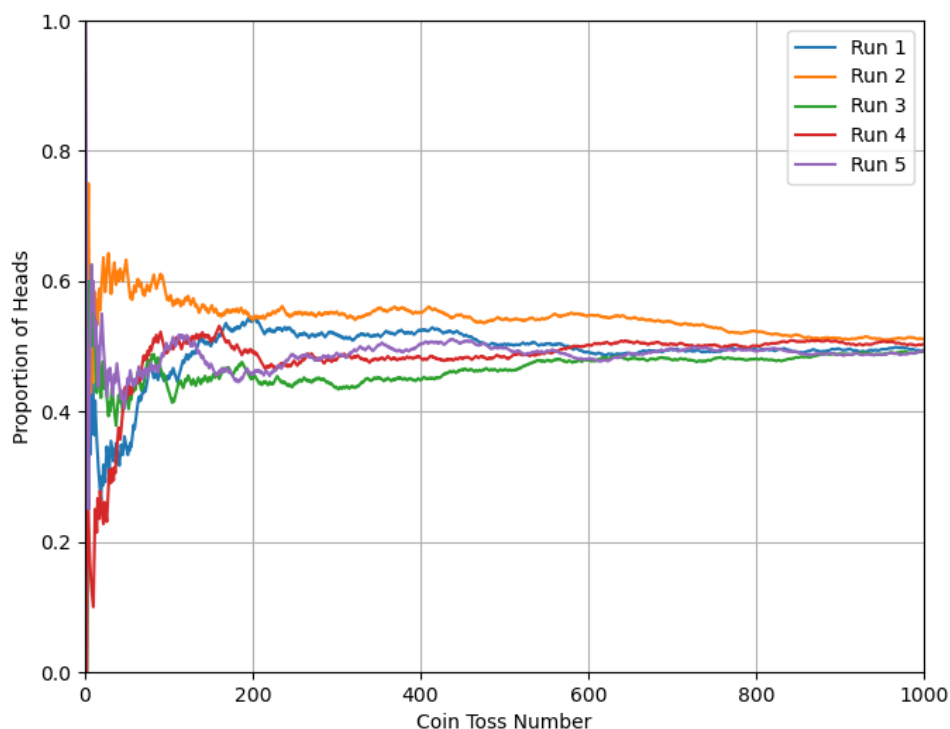
for rep in range(5):      # repeat simulation 5 times
    toss = (rng.random(N) < p)

    cum_avg = []
    running_sum = 0

    for i in range(1, N+1):
        running_sum = running_sum + toss[i-1]
        avg_i = running_sum / i
        cum_avg.append(avg_i)

    # plot this repetition
    plt.plot(range(1, N+1), cum_avg, label=f"Run {rep+1}")

plt.xlabel("Coin Toss Number")
plt.ylabel("Proportion of Heads")
plt.axis([0, N, 0, 1])
plt.legend()
plt.grid(True)
```



Generating Arbitrary Discrete Distributions from Uniform(0,1)

Method: Inverse Transform for Discrete Distributions

Specify the random variable X

We want to construct a random variable X such that:

$$P(X = x_j) = p_j, \quad j = 0, \dots, n, \quad \sum_{j=0}^n p_j = 1.$$

Build the cumulative distribution of X (CDF)

$$F(x) = \begin{cases} 0 & x < x_0 \\ p_0 & x_0 \leq x < x_1 \\ p_0 + p_1 & x_1 \leq x < x_2 \\ \vdots & \end{cases}$$

This gives a monotone increasing sequence:

$$0 < F(x_0) < F(x_1) < \dots < F(x_n) = 1.$$

Partition the interval $[0, 1]$

Use the CDF values to divide $[0, 1]$:

$$[0, F(x_0)), \quad [F(x_0), F(x_1)), \quad \dots, \quad [F(x_{n-1}), 1).$$

Each interval has length:

$$F(x_j) - F(x_{j-1}) = p_j.$$

Generating Discrete Distributions from Uniform(0,1)

Generate uniform random numbers

$$U \sim \text{Uniform}(0, 1).$$

Define X via U

Define:

$$X = x_j \quad \text{if} \quad F(x_{j-1}) \leq U < F(x_j)$$

Or as “Inverse” CDF

$$X = F^{-1}(U)$$

Defined by

$$F^{-1}(u) = \begin{cases} x_0 & 0 \leq u < F(x_0) \\ x_1 & F(x_0) \leq u < F(x_1) \\ x_2 & F(x_1) \leq u < F(x_2) \\ \vdots & \end{cases}$$

Rule for generating
values that are
distributed like X

Verify construction

$$P(X = x_j) = P(F(x_{j-1}) \leq U < F(x_j)) = F(x_j) - F(x_{j-1}) = p_j.$$

Generating Discrete Distributions from Uniform(0,1)

Example: Specify a distribution from the jumps

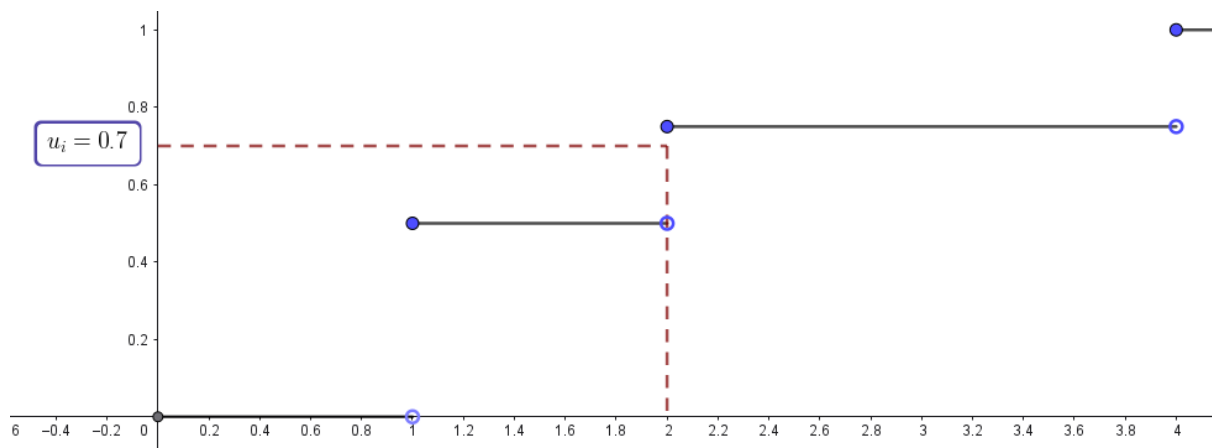
$$P(X = 1) = 0.5, \quad P(X = 2) = 0.25, \quad P(X = 4) = 0.25$$

Build the CDF

$$F(1) = 0.5, \quad F(2) = 0.75, \quad F(4) = 1$$

General rule:

$$X = x_j \quad \text{if } F(x_{j-1}) \leq U < F(x_j)$$



$U_i = 0.7$ satisfies

$$0.5 = F(1) < 0.7 \leq F(2) = 0.75.$$

For the inverse CDF we define

$$F_X^{-1}(u) = \min\{x : F_X(x) \geq u\}.$$

So the x -value corresponding to $U_i = 0.7$ is

$$x = 2.$$

Generating Discrete Distributions from Uniform(0,1)

Example: simulate a value of a random variable X such that

$$P(X = 1) = 0.5, \quad P(X = 2) = 0.25, \quad P(X = 4) = 0.25$$

Python Implementation

```
import numpy as np

rng = np.random.default_rng()

# Step 1: Define values and probabilities (Example 3)
values = [1, 2, 4]
p = [0.5, 0.25, 0.25]

# Step 2: Build cumulative probabilities (CDF values)
A = np.cumsum(p)  # [0.5, 0.75, 1.0]

# Step 3: Use the same U as in the example
U = 0.7

# Step 4: Find the interval containing U
j = 0
while U > A[j]:
    j = j + 1

# Step 5: Assign X
X = values[j]

print("U =", U)
print("CDF boundaries:", A)
print("Sampled X =", X)
```

```
U = 0.7
CDF boundaries: [0.5  0.75  1.  ]
Sampled X = 2
```

Generating Discrete Distributions from Uniform(0,1)

Example 5. simulate 10000 values of a random variable X such that

$$P(X = 1) = 0.5, \quad P(X = 2) = 0.25, \quad P(X = 4) = 0.25$$

Python Implementation

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng()

# Step 1: Define values and probabilities (Example 3)
values = [1, 2, 4]
p = [0.5, 0.25, 0.25]

# Step 2: Build cumulative probabilities (CDF)
A = np.cumsum(p) # [0.5, 0.75, 1.0]

# Number of samples
N = 10000
samples = []

# Step 3: Generate samples using inverse transform
for _ in range(N):
    U = rng.random()

    j = 0
    while U > A[j]:
        j = j+1

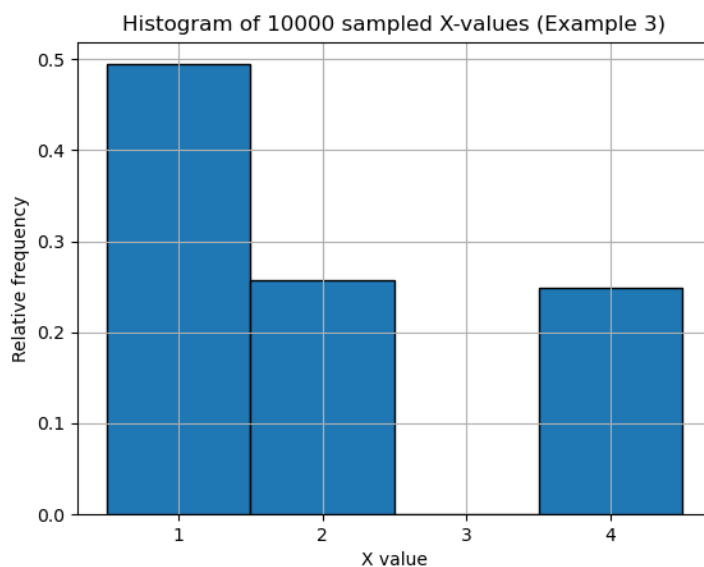
    X = values[j]
    samples.append(X)

# Step 4: Plot histogram
plt.hist(samples,
        bins=[0.5, 1.5, 2.5, 3.5, 4.5],
        edgecolor='black',
        density=True)

plt.xticks([1, 2, 3, 4])
plt.xlabel("X value")
plt.ylabel("Relative frequency")
plt.title("Histogram of 10000 sampled X-values (Example 3)")
plt.grid(True)
plt.show()
```

The PMF describes the distribution.

The inverse CDF lets you **generate samples from it**.



General transformation framework – continuous variables

Recall transformation by **increasing** function g , such that $Y = g(X)$.

Then

$$F_Y(y) = P(Y \leq y) = P(g(X) \leq y) = P(X \leq g^{-1}(y)) = F_X(g^{-1}(y))$$

Special case: Probability Integral Transform

Now choose a *very specific* transformation:

$$Y = g(X) = F_X(X)$$

X transformed by
its own CDF!

Then:

$$F_Y(y) = F_X(F_X^{-1}(y)) = y$$

So:

$$Y = F_X(X) \sim \text{Uniform}(0, 1)$$

Generating Arbitrary Continuous Distributions from Uniform(0,1)

Method: Inverse Transform for continuous Distributions

To apply the inverse transform method, we must know the **target distribution** of X , i.e.:

- either the **CDF** $F_X(x)$, or
- a **PDF** $f_X(x)$ from which we can compute $F_X(x)$

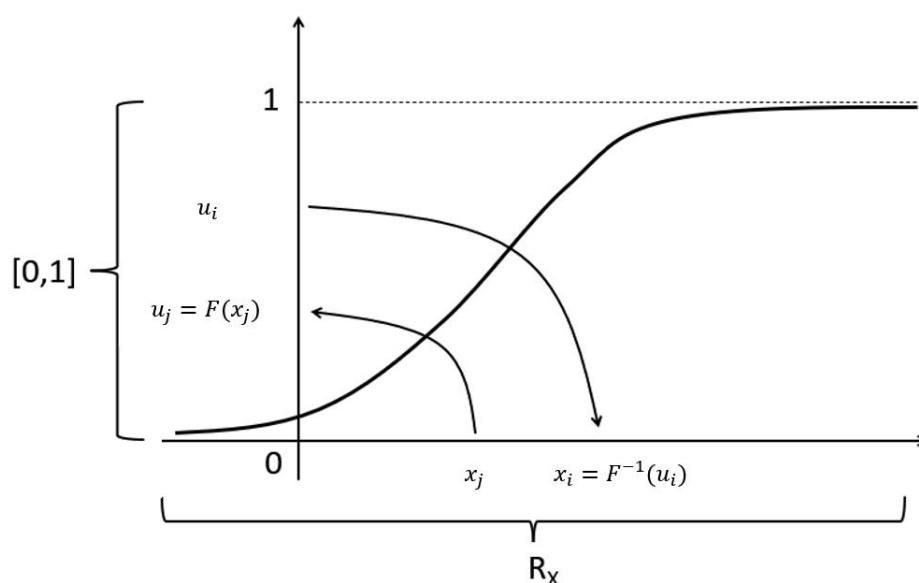
Now use Probability Integral Transform

Since Y is uniform we write U instead:

$$U = F_X(X)$$

Since $F_X(X) \sim \text{Uniform}(0, 1)$, we can reverse this transformation to generate X from a uniform random variable.

$$U \sim \text{Uniform}(0, 1), \quad X = F_X^{-1}(U)$$



Generating Continuous Distributions from Uniform(0,1)

Example 6. Generating Exponential Random Variables

The exponential distribution with parameter $\lambda = 1$ has CDF:

$$F(x) = 1 - e^{-x}, \quad x > 0.$$

Using the **inverse transform method**:

$$X = F^{-1}(U). \quad \leftarrow U \sim \text{Uniform}(0, 1)$$

Solve for X :

$$\begin{aligned} u &= 1 - e^{-x} \\ e^{-x} &= 1 - u \\ x &= -\ln(1 - u). \end{aligned} \quad \leftarrow \text{The inverse CDF (function)}$$

Thus,

$$X = -\ln(1 - U). \quad \leftarrow \text{Random variable generation by use of } U$$

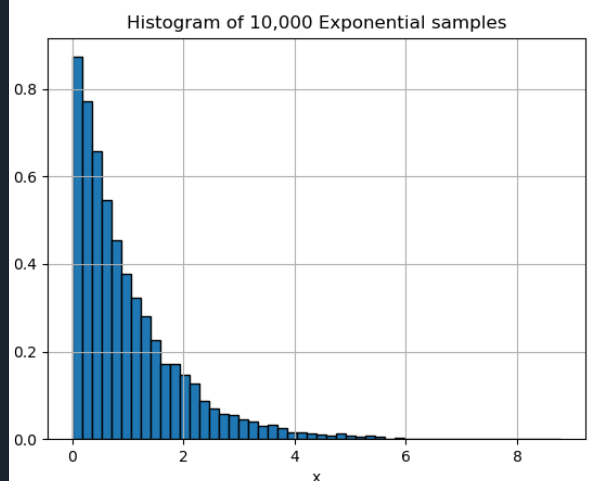
Python Implementation

```
rng = np.random.default_rng()

N = 10000
samples = []
for k in range(N):
    U = rng.random()
    X = -np.log(1-U)
    samples.append(X)

print("Generated", N, "Exponential(1) samples.")

plt.hist(samples, bins=50, density=True, edgecolor='black')
plt.xlabel("x")
plt.ylabel("density")
plt.title("Histogram of 10,000 Exponential(1) samples")
plt.grid(True)
plt.show()
```



What the histogram computes: **hist(x, bins=50, density=True)**

The histogram with `density=True` plots

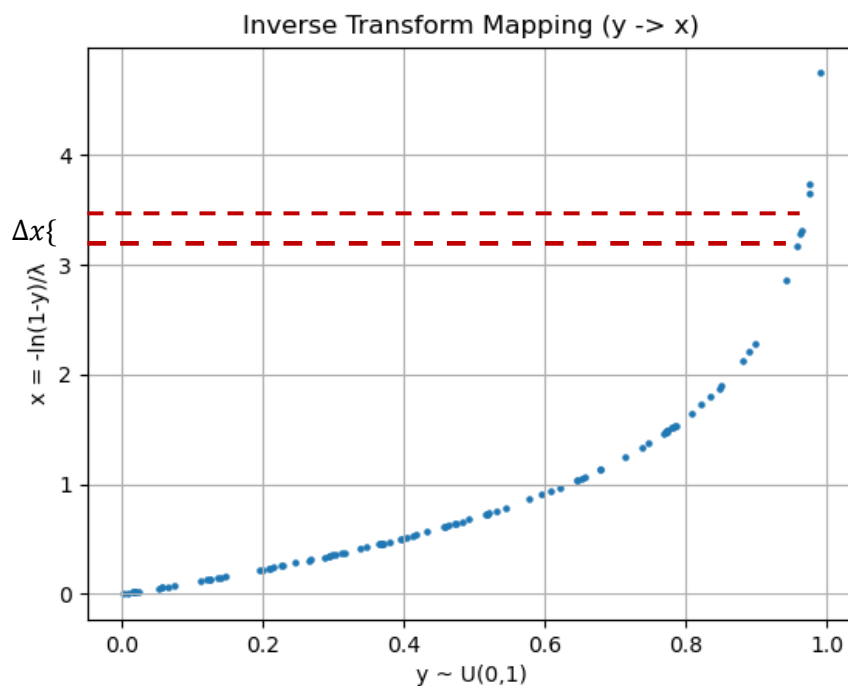
$$\frac{\text{count in bin}}{N \Delta x}$$

Where

- N = total number of samples you generated.
- Bin = x-axis range $[\min(x), \max(x)]$ is divided into **50 intervals**.
- Δx (bin width) = $\frac{x_{\max} - x_{\min}}{N_{\text{bins}}}$

Without density=True (normalization), the bar height would simply be *count*

With density=True, matplotlib divides by N and Δx



$$\begin{aligned} \frac{\text{count in bin}}{N} &\approx P(x < X < x + \Delta x) = P(F(x) < U < F(x + \Delta x)) \\ &= \int_{F(x)}^{F(x+\Delta x)} f_U(u) du = \int_{F(x)}^{F(x+\Delta x)} 1 du = F(x + \Delta x) - F(x) \approx F'(x) \Delta x = f(x) \Delta x \end{aligned}$$

As Δx becomes smaller the approximation becomes better:

$$\frac{\text{count in bin}}{N} = f(x) \Delta x \Rightarrow \frac{\text{count in bin}}{N \Delta x} = f(x)$$

Apply it to the exponential

For exponential:

$$F(x) = 1 - e^{-x}$$

so

$$P(x \leq X \leq x + \Delta x) = e^{-x} - e^{-(x+\Delta x)}$$

Factor:

$$= e^{-x}(1 - e^{-\Delta x})$$

For small Δx :

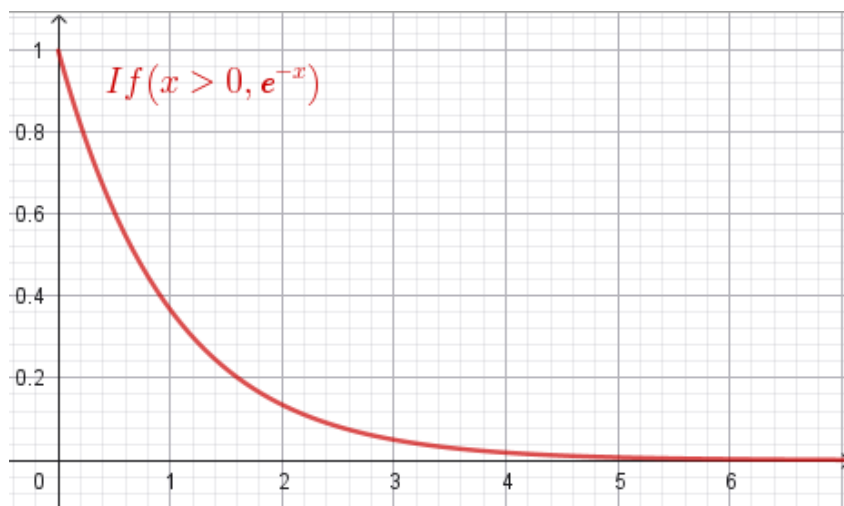
$$1 - e^{-\Delta x} \approx \Delta x$$

Thus

$$P(x \leq X \leq x + \Delta x) \approx e^{-x} \Delta x$$

Divide by Δx :

$$f(x) = e^{-x}$$



Box–Muller Transform

The standard normal distribution is extremely important in statistics, but its CDF has **no closed-form**. So we cannot use the "inverse CDF method" directly. Instead, we use a clever **uniform-to-normal** transformation.

Problem: The 1D standard normal PDF (gaussian)

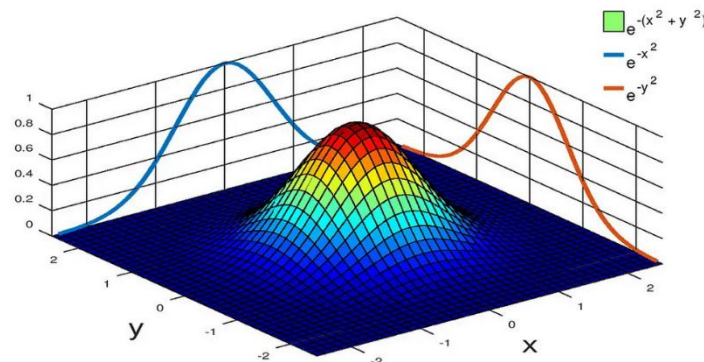
$$f(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$$

This shape is hard to generate directly from Uniform(0,1).

There is no simple transformation $X = F_X^{-1}(U)$ that turns a single uniform random number into a Gaussian.

Solution: Exploit the circular symmetry of the joint 2D Gaussian

$$f(x, y) = \frac{1}{2\pi} e^{-(x^2+y^2)/2}$$



We want to generate:

$$(Z_0, Z_1) \sim \text{2D Gaussian}$$

using **independent uniforms**:

$$U_1, U_2 \sim \text{Uniform}(0, 1)$$

GOAL: Break the problem into **two independent pieces we can generate separately**

Course: Stokastisk modellering og behandling (SWSTS-01)

2D Gaussian depends only on the **distance from the origin**, not on the direction:

$$x^2 + y^2 = r^2.$$

In **polar coordinates**, the density becomes:

$$f(r, \theta) = \frac{1}{2\pi} r e^{-r^2/2}, \quad r \geq 0, 0 \leq \theta < 2\pi.$$

Find the marginal of angle θ :

$$f_{\Theta}(\theta) = \int_0^{\infty} f(r, \theta) dr.$$

Plug in the joint density

$$f_{\Theta}(\theta) = \frac{1}{2\pi} \int_0^{\infty} r e^{-r^2/2} dr.$$

Evaluate the integral

$$f_{\Theta}(\theta) = \frac{1}{2\pi} \cdot 1 = \frac{1}{2\pi}.$$

Notice that **θ does not appear anywhere in the formula** because of rotational symmetry - Every angle is equally likely.

Since the marginal density of θ is **constant**, it is a **uniform distribution** on $[0, 2\pi)$.

$$\Theta \sim \text{Uniform}(0, 2\pi).$$

Find the marginal of radius R:

$$f_R(r) = \int_0^{2\pi} f_{R,\Theta}(r, \theta) d\theta.$$

Plug in the joint density

$$f_R(r) = r e^{-r^2/2} \int_0^{2\pi} \frac{1}{2\pi} d\theta.$$

Evaluate the integral

$$\int_0^{2\pi} \frac{1}{2\pi} d\theta = \frac{1}{2\pi} \cdot 2\pi = 1.$$

So

$$f_R(r) = r e^{-r^2/2}, \quad r \geq 0.$$

This is not a standard distribution we can sample directly. So we use the **inverse transform method**.

1: Compute the CDF of R

$$F_R(r) = \int_0^r s e^{-s^2/2} ds = 1 - e^{-r^2/2}$$

2: Apply the probability integral transform

$$U_1 = F_R(R) \sim \text{Uniform}(0, 1) \Rightarrow R = F_R^{-1}(U_1)$$

3: Solve for R

$$u = 1 - e^{-r^2/2} \Rightarrow e^{-r^2/2} = 1 - u \Rightarrow r = \sqrt{-2 \ln(1 - u)}$$

Thus:

$$R = \sqrt{-2 \ln(1 - U_1)}$$

Since $1 - U_1 \sim \text{Uniform}(0, 1)$, we typically write:

$$R = \sqrt{-2 \ln U_1}$$

Θ and R are independent because:

$$f_{R,\Theta}(r, \theta) = f_{\Theta}(\theta) f_R(r).$$

Generate R and Θ from uniforms

Take two independent uniforms:

$$U_1, U_2 \sim \text{Uniform}(0, 1) \quad \text{i.i.d.}$$

Then define

$$R = \sqrt{-2 \ln U_1}, \quad \Theta = 2\pi U_2.$$

Convert back to Cartesian coordinates

Now use the polar-to-Cartesian formulas:

$$Z_0 = R \cos \Theta, \quad Z_1 = R \sin \Theta.$$

Substituting the expressions for R and Θ ,

$$Z_0 = \sqrt{-2 \ln U_1} \cos(2\pi U_2),$$

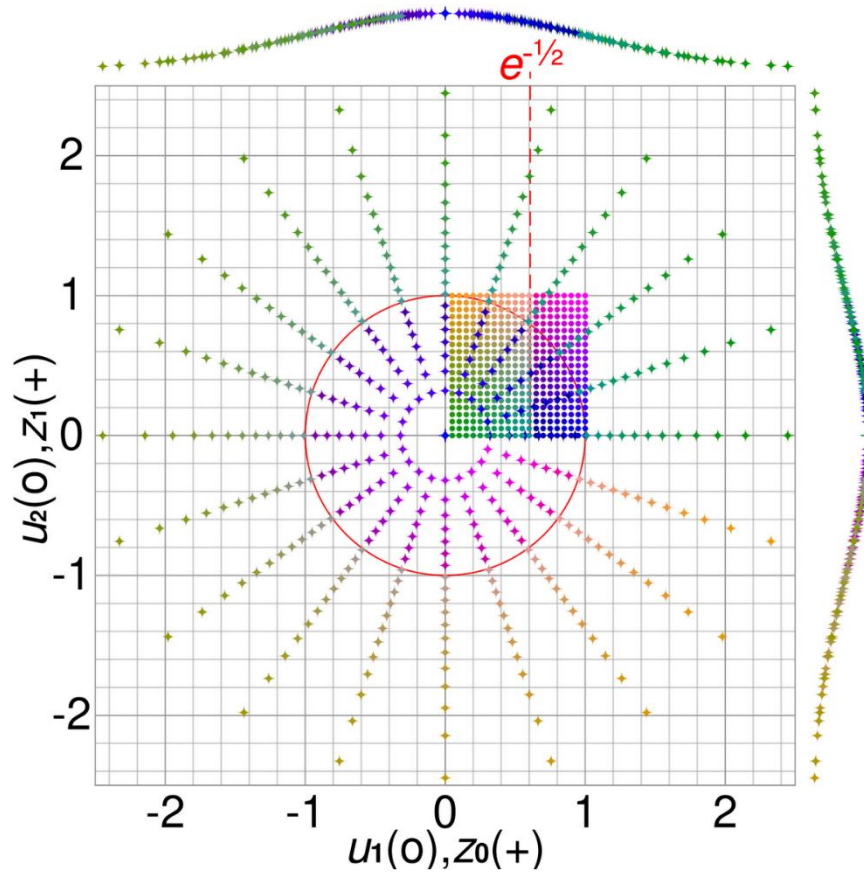
$$Z_1 = \sqrt{-2 \ln U_1} \sin(2\pi U_2).$$

These are the **Box-Muller formulas**.

$$Z_0 = \sqrt{-2 \ln U_1} \cos(2\pi U_2), \quad Z_1 = \sqrt{-2 \ln U_1} \sin(2\pi U_2)$$

And the result is:

$$Z_0, Z_1 \sim N(0, 1) \quad \text{independently.}$$

Visualisation of the Box–Muller transform:

- The coloured points in the unit square (u_1, u_2) , are mapped to a 2D Gaussian (z_0, z_1) .
- The plots at the margins are the probability distribution functions of Z_0 and Z_1 .

Example 7. Generating pairs of normal random variables**Python Implementation**

```

rng = np.random.default_rng()

U = rng.random((2, 5000))    #Vectorized array

# Apply the Box-Muller formulas
R = np.sqrt(-2 * np.log(U[0])) # radius component
Z1 = R * np.cos(2 * np.pi * U[1])
Z2 = R * np.sin(2 * np.pi * U[1])

plt.hist(Z1, bins=30, alpha=0.6, density=True, label="Z1")
plt.hist(Z2, bins=30, alpha=0.6, density=True, label="Z2")
plt.legend()
plt.title("Histograms of Box-Muller Normal Samples")
plt.show()

```

